MÓDULO 2-ALGORITMOS EN R*

Cañadas Ian[1], Bresci, Joaquin[2], Isgro, Mateo[1], La Rocca, Nicolás[1], Salvo, Luca[1], and Zanetti, Bautista[1]

[1] Universidad Nacional de Cuyo, Instituto de Ingenería Industrial,POBOX 5502,C.P.:5500 Mendoza , Argentina
2

lncs@uncuyo.com
http://www.uncuyo.edu.ar/contacto
3

Abstract. R Posit en un software online que permite utilizar lenguaje R y trabajar tambien con matrices de forma similar a Mathlab

Keywords: Lenguaje R[1–3] · TYHM · Algoritmos

1. Introducción

Con R podemos transformar problemas del día a día en modelos matemáticos. Lo interesante es que no nos quedamos con la primera solución, sino que comparamos varios algoritmos para ver cuál es más rápido y consume menos recursos

2.1. Ejercicio 1: Generar un vector secuencia

```
knitr::opts_chunk$set(echo = TRUE)
# Definición de variables y operaciones básicas
a <- 40
b <- 60
c <- b - a
print(c)
```

```
## [1] 20
```

2.1.2. Creación de una matriz de 3x3

Escribimos 'ncol' para cantidad de columnas Siempre es byrow TRUE o FALSE para que lo acomode en orden de filas o de columnas

```
# Crea una matriz con números del 1 al 9
mi_matriz <- matrix(1:9, nrow = 3, ncol = 3)

# Ver el resultado
print(mi_matriz)
```

* Supported by organization x.

```
##      [,1] [,2] [,3]
## [1,]    1    4    7
## [2,]    2    5    8
## [3,]    3    6    9
```

2.3 Uso de base de datos

```r
knitr::opts_chunk$set(echo = TRUE)
# Resumen de base de datos interna
summary(cars)
```

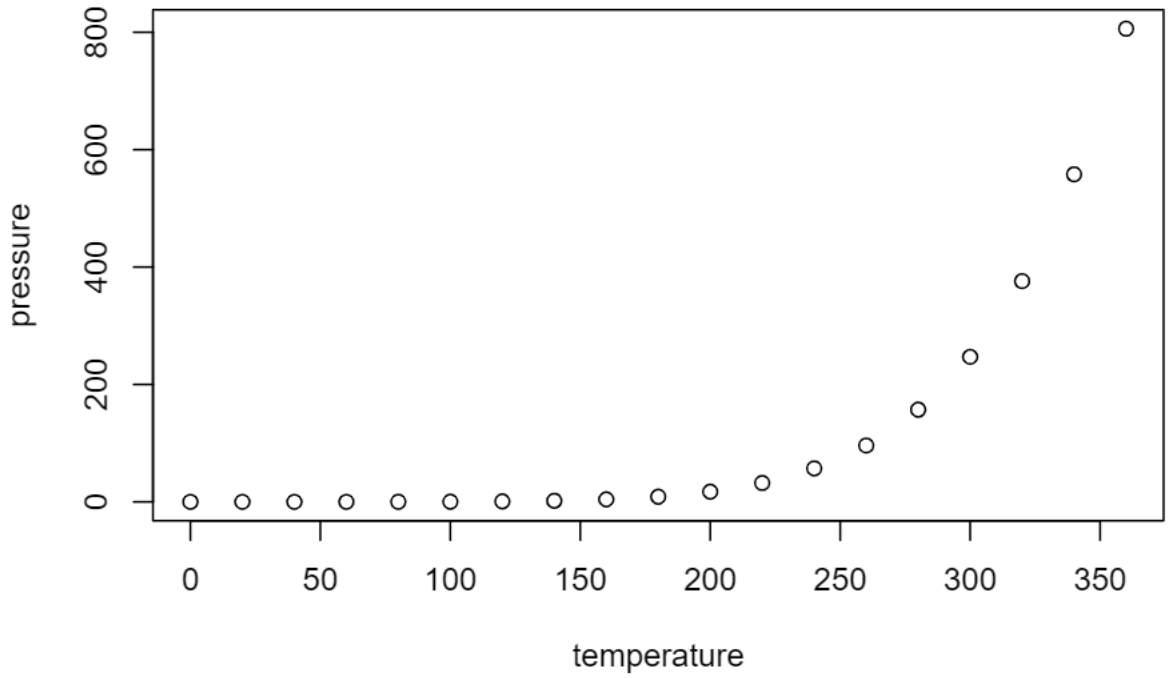
```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00  
```

```r
print(cars)
```

```
##    speed dist
## 1      4    2
## 2      4   10
## 3      7    4
## 4      7   22
## 5      8   16
## 6      9   10
## 7     10   18
## 8     10   26
## 9     10   34
## 10    11   17
## 11    11   28
## 12    12   14
## 13    12   20
## 14    12   24
## 15    12   28
## 16    13   26
## 17    13   34
## 18    13   34
## 19    13   46
## 20    14   26
## 21    14   36
## 22    14   60
## 23    14   80
```

```
## 24     15    20
## 25     15    26
## 26     15    54
## 27     16    32
## 28     16    40
## 29     17    32
## 30     17    40
## 31     17    50
## 32     18    42
## 33     18    56
## 34     18    76
## 35     18    84
## 36     19    36
## 37     19    46
## 38     19    68
## 39     20    32
## 40     20    48
## 41     20    52
## 42     20    56
## 43     20    64
## 44     22    66
## 45     23    54
## 46     24    70
## 47     24    92
## 48     24    93
## 49     24   120
## 50     25    85
```

```r
plot(pressure)
```

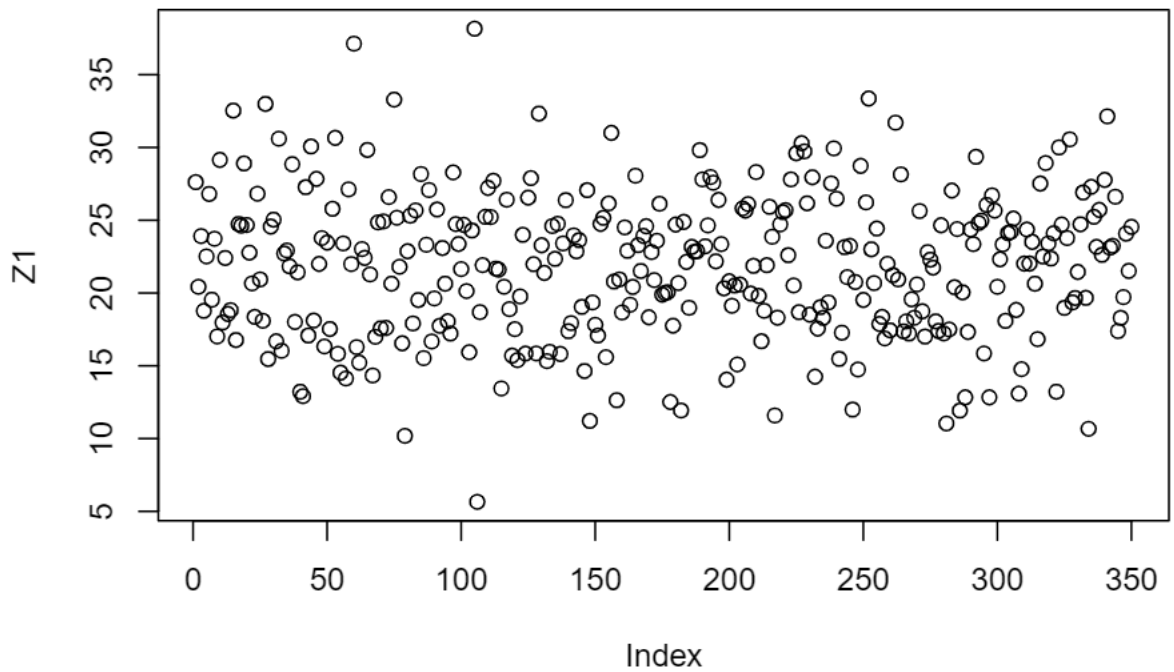



```r
knitr::opts_chunk$set(echo = TRUE)
 # Generar 350 datos con media 22 y desviación estándar 5
Z1 <- rnorm(350, 22, 5)
print(Z1)
```

```
##   [1] 27.616124 20.428431 23.910027 18.769403 22.512413 26.804320 19.540034
##   [8] 23.728227 17.011779 29.145023 17.971132 22.403383 18.562476 18.823220
##  [15] 32.531534 16.770189 24.741601 24.624157 28.899862 24.670671 22.772071
##  [22] 20.657226 18.359337 26.817833 20.928573 18.090036 32.989888 15.463566
##  [29] 24.561922 25.041113 16.681775 30.599444 16.028263 22.712439 22.920768
##  [36] 21.833373 28.845905 18.010769 21.418944 13.216667 12.912444 27.272607
##  [43] 17.077735 30.065672 18.108664 27.838389 21.995031 23.771554 16.326003
##  [50] 23.464435 17.526940 25.783226 30.656638 15.824005 14.528664 23.408706
##  [57] 14.128848 27.121132 21.985568 37.129777 16.284261 15.217229 23.018373
##  [64] 22.411969 29.827510 21.278471 14.330567 16.991140 24.850819 17.571373
##  [71] 24.919077 17.597982 26.589135 20.638304 33.284629 25.172172 21.799838
##  [78] 16.543828 10.188261 22.870034 25.310888 17.905603 25.673414 19.517527
##  [85] 28.171481 15.524435 23.315514 27.072153 16.668791 19.624996 25.735200
##  [92] 17.743410 23.093456 20.638386 18.055554 17.211818 28.281566 24.735455
##  [99] 23.368172 21.642648 24.676370 20.128127 15.932811 24.292634 38.154696
## [106]  5.652543 18.685170 21.912888 25.229607 27.220071 25.215703 27.705126
## [113] 21.672353 21.608429 13.440798 20.432592 26.406558 18.889618 15.696744
## [120] 17.513204 15.389178 19.762693 23.996558 15.839651 26.552391 27.891720
## [127] 21.980470 15.843764 32.326670 23.281541 21.378141 15.314502 15.955559
```

```
## [134] 24.595056 22.344405 24.758626 15.809687 23.409106 26.377690 17.372491
## [141] 17.931689 23.958814 22.861274 23.627050 19.046407 14.640252 27.065183
## [148] 11.212595 19.351960 17.818719 17.074884 24.749025 25.164165 15.594378
## [155] 26.152038 31.002443 20.785417 12.633958 20.930444 18.661959 24.510537
## [162] 22.893379 19.190652 20.417891 28.057097 23.284126 21.511819 23.919092
## [169] 24.584466 18.331844 22.810000 20.883218 23.585408 26.127618 19.890091
## [176] 20.004423 20.050467 12.514351 17.754591 24.702607 20.681765 11.930676
## [183] 24.885055 22.132928 18.983823 23.164764 22.829113 22.873468 29.812725
## [190] 27.806189 23.209240 24.641166 27.968308 27.576844 22.174974 26.396160
## [197] 23.357923 20.321257 14.045626 20.805221 19.123248 20.511543 15.088785
## [204] 20.581199 25.813420 25.656862 26.108074 19.960267 21.865795 28.310551
## [211] 19.799158 16.688302 18.778180 21.911324 25.921024 23.860509 11.586432
## [218] 18.303863 24.714202 25.551087 25.687847 22.596087 27.801161 20.516475
## [225] 29.598320 18.681309 30.289749 29.733046 26.162724 18.515802 27.949262
## [232] 14.253112 17.554983 19.024289 18.290879 23.592896 19.351888 27.534716
## [239] 29.937461 26.481097 15.472389 17.271340 23.148732 21.088023 23.236481
## [246] 11.988917 20.755929 14.744425 28.734776 19.523568 26.222904 33.363221
## [253] 23.001104 20.680290 24.428355 17.870662 18.358807 16.873200 22.006088
## [260] 17.433366 21.224151 31.699624 20.934327 28.149260 17.363505 18.045553
## [267] 17.212568 19.568374 18.298137 20.584039 25.631956 18.743575 17.008544
## [274] 22.812447 22.290115 21.742870 18.052685 17.390442 24.655367 17.222686
## [281] 11.037838 17.500087 27.036342 20.398728 24.392991 11.919544 20.045112
## [288] 12.831776 17.325341 24.357602 23.359701 29.356729 24.805085 24.970063
## [295] 15.850319 26.049131 12.834806 26.700050 25.661638 20.428011 22.318393
## [302] 23.335897 18.085640 24.109069 24.208912 25.110366 18.837416 13.090691
## [309] 14.764434 22.021309 24.360681 22.021147 23.506579 20.639377 16.827706
## [316] 27.517196 22.506642 28.924386 23.408093 22.368037 24.100536 13.218457
## [323] 29.999386 24.700908 18.978615 23.775606 30.552382 19.356179 19.656129
## [330] 21.447855 24.720773 26.904046 19.664685 10.672749 27.292015 25.231538
## [337] 23.167705 25.710680 22.618452 27.769227 32.135391 23.105620 23.248091
## [344] 26.611386 17.379492 18.289283 19.715629 24.076622 21.514760 24.541537
```

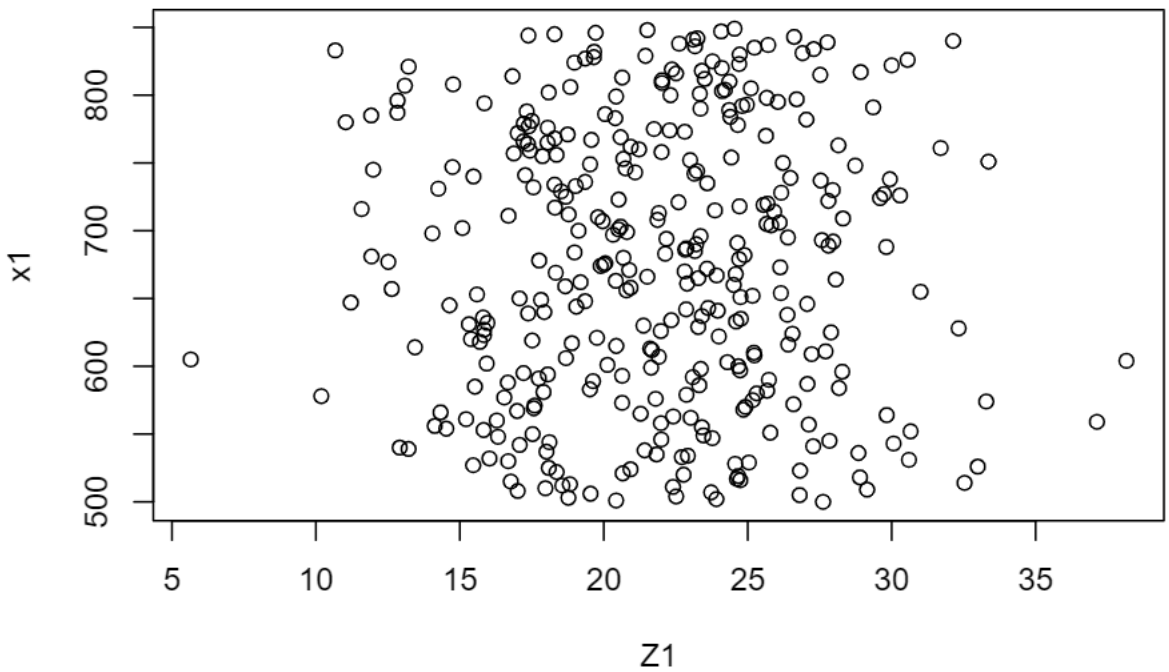
```r
# Graficar los puntos
plot(Z1)
```



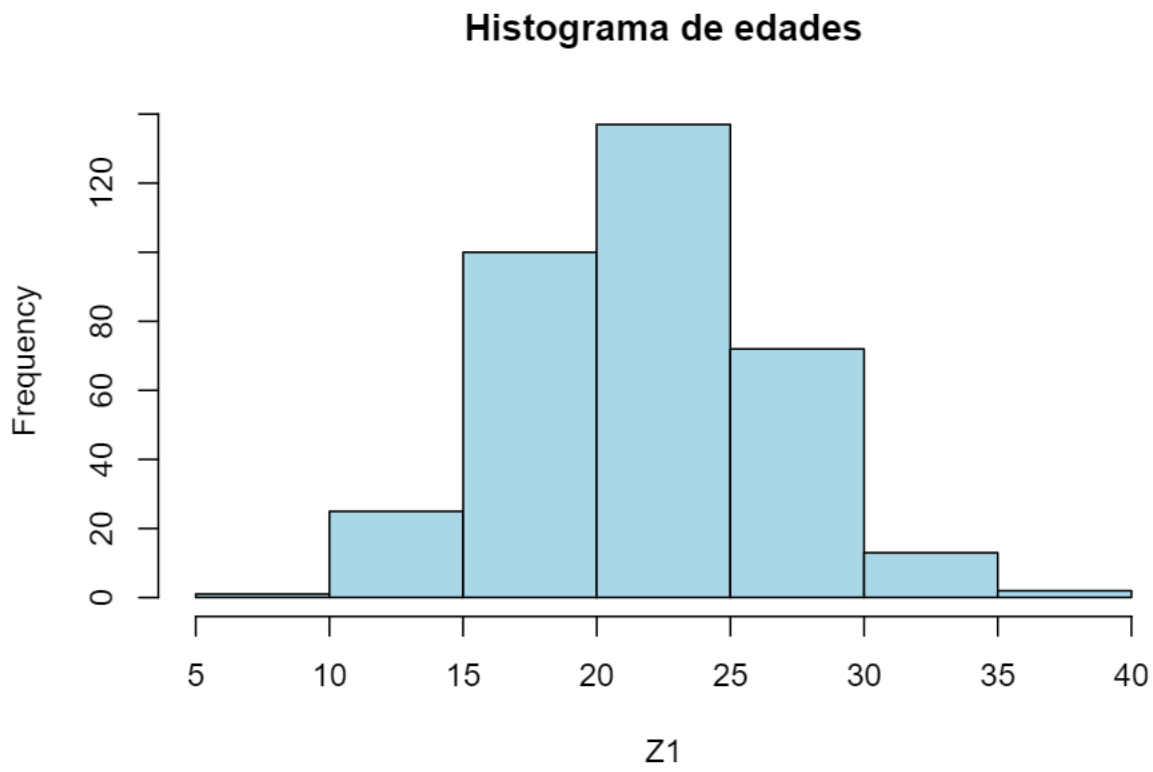

```r
# Obtener la longitud
w1 <- length(Z1)
print(w1)
```

```
## [1] 350
```

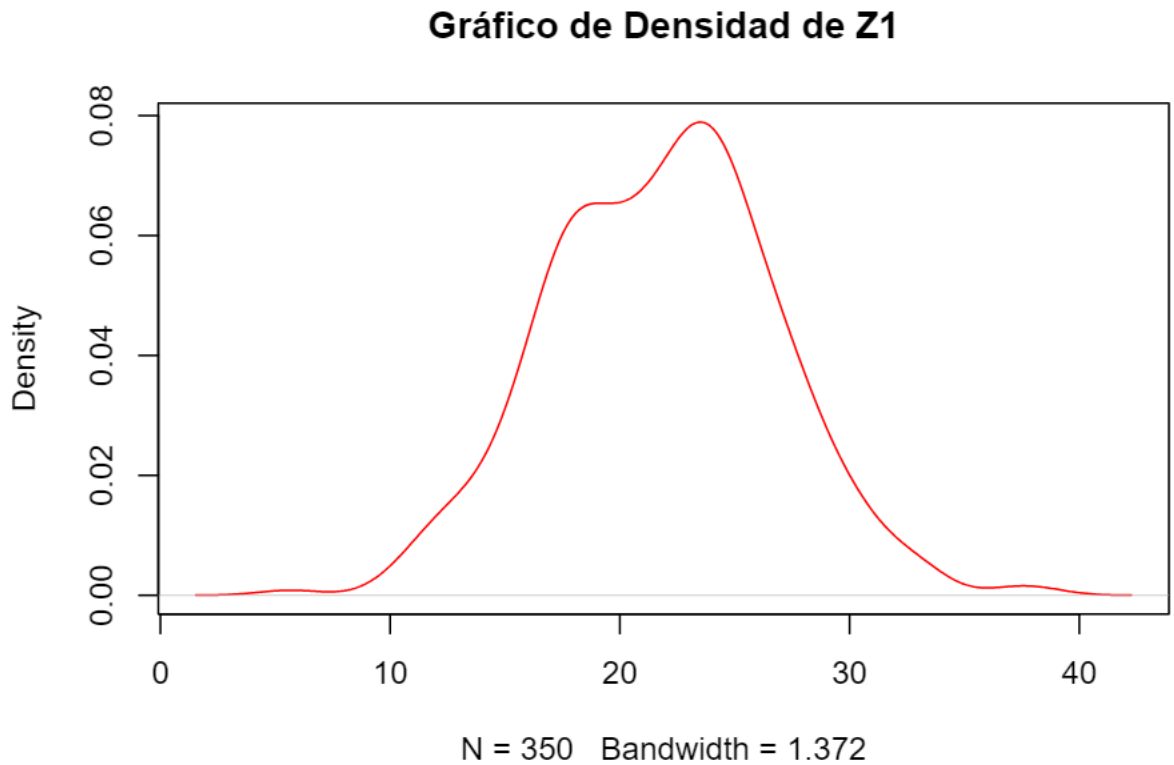
```r
# Crear una secuencia de la misma longitud para graficar (350 elementos)
x1 <- seq(500, length.out = 350)
plot(Z1, x1)
```




```r
# Creación de histograma
hist(Z1, main = "Histograma de edades", breaks = 10, col = "lightblue")
```




```r
# Cálculo y gráfico de densidad
densidad <- density(Z1)
plot(densidad, main = "Gráfico de Densidad de Z1", col = "red")
```



0.1 Ejercicio 1

DNI—910

```
#CONSIGNA: CREAR UN VECTOR QUE TENGA TODOS LOS NUMEROS ENTEROS DESDE EL 1 AL 910.
secuencia_dni<-(1:447)
print(secuencia_dni)
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
```

```
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285 286 287 288
## [289] 289 290 291 292 293 294 295 296 297 298 299 300 301 302 303 304 305 306
## [307] 307 308 309 310 311 312 313 314 315 316 317 318 319 320 321 322 323 324
## [325] 325 326 327 328 329 330 331 332 333 334 335 336 337 338 339 340 341 342
## [343] 343 344 345 346 347 348 349 350 351 352 353 354 355 356 357 358 359 360
## [361] 361 362 363 364 365 366 367 368 369 370 371 372 373 374 375 376 377 378
## [379] 379 380 381 382 383 384 385 386 387 388 389 390 391 392 393 394 395 396
## [397] 397 398 399 400 401 402 403 404 405 406 407 408 409 410 411 412 413 414
## [415] 415 416 417 418 419 420 421 422 423 424 425 426 427 428 429 430 431 432
## [433] 433 434 435 436 437 438 439 440 441 442 443 444 445 446 447
```

```
## Ejercicio 2
##Calcular la suma de todos los valores del vector secuencia_dni
Total<-0
valor_final<-length(secuencia_dni)
for (i in 1:valor_final)
Total<-Total+i
Total
```

```
## [1] 100128
```

0.2 Ejercicio 3

#Repetir ejercicio 2 pero usando Python
valor_final = length(secuencia_dni) total = 0
for i in range(1, valor_final + 1): total += i
print(total)

0.3 Ejercicio 4

#cuanto demora en correr el código del ejercicio 2

```
# Registramos el momento de inicio
inicio <- Sys.time()

Total <- 0
 'length'
```

```
## [1] "length"
```

```
valor_final <- length(secuencia_dni)

for (i in 1:valor_final) {
```

```
  Total <- Total + i
}

# Registramos el momento de finalización
fin <- Sys.time()

# Calculamos la diferencia
duracion <- fin - inicio

# Mostramos los resultados
print(Total)
```

```
## [1] 100128
```

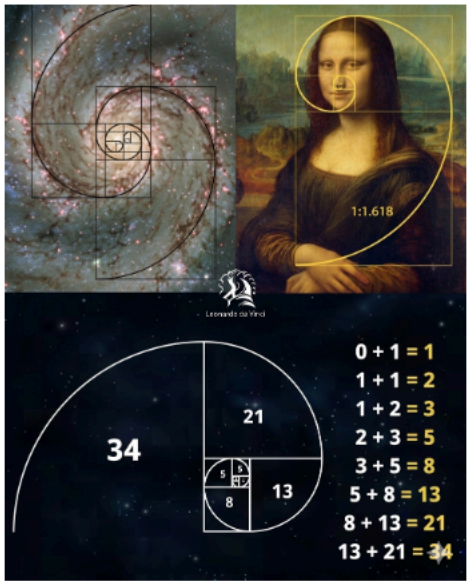
```
print(duracion)
```

```
## Time difference of 0.004957676 secs
```

0.4 Ejercicio 5

1 Implementación de una serie Fibonacci

La sucesión de Fibonacci es una serie infinita de números naturales donde cada número se obtiene sumando los dos anteriores. Es una de las secuencias más famosas de las matemáticas porque aparece constantemente en la naturaleza (piñas,girasoles,ramas de árboles,caracoles,etc)



La serie Fibonacci comienza con los números 0 y 1, a partir de estos cada uno de los siguientes términos es la suma de los dos anteriores.(0,1,1,2,3,5,8,13...) A continuación puede verse el código para implementar la serie:

```
F <- vector(length = 100)
A <- 0
B <-1
F[1] <-A
F[2] <-B
for (i in 3:100)
{F[i] <-(F[i-1]+F[i-2])}
head(F)
```

```
## [1] 0 1 1 2 3 5
```

1.1 Ejercicio 6

La **Ordenación de burbuja** (Bubble Sort en inglés) es un sencillo algoritmo de ordenamiento. Funciona revisando cada elemento de la lista que va a ser ordenada con el siguiente, intercambiándolos de posición si están en el orden equivocado. Es necesario revisar varias veces toda la lista hasta que no se necesiten más intercambios, lo cual significa que la lista está ordenada. Este algoritmo obtiene su nombre de la forma con la que suben por la lista los elementos durante los intercambios, como si fueran pequeñas burbujas. También es conocido como el método del intercambio directo. Dado que solo usa comparaciones para operar elementos, se lo considera un algoritmo de comparación, siendo uno de los más sencillos de implementada.

```
# 1. Definición de la función (Método Burbuja)
burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n - 1)) {
    for (i in 1:(n - j)) {
      if (x[i] > x[i + 1]) {
        temp <- x[i]
        x[i] <- x[i + 1]
        x[i + 1] <- temp
      }
    }
  }
  return(x)
}
```

1.2 Ejercicio 7

Compara la performance de ordenación del método burbuja vs el método sort de R Usar método microbenchmark para una muestra de tamaño 20.000

```r
# 2. Comparación de rendimiento
library(microbenchmark)

# Generamos un vector de prueba
set.seed(123) # Para que los resultados sean replicables
test_data <- sample(1:1000, 100)

comparativa <- microbenchmark(
  Metodo_Burbuja = burbuja(test_data),
  Funcion_Base_R = sort(test_data),
  times = 50
)

print(comparativa)
```

```
## Unit: microseconds
##            expr     min      lq      mean   median      uq      max neval
##  Metodo_Burbuja 536.811 543.492 686.68386 547.6470 562.762 7119.722    50
##  Funcion_Base_R  25.960  27.780  37.17332  30.2805  39.030  213.591    50
```

1.3 Ejercicio 8

La penitencia de Newton Comentan algunos historiadores que Newton tenía en le escuela primaria un profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que el estaba enseñando. Tan pronto alguien cometía un acto improcedente lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma de desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado previamente y que tenía registrado en un voluminoso cuaderno que (al esltio tabla de logaritmos) cargaba religiosamente en su portafolios de cuero. Consigna de trabajo : Desarrolla dos algoritmos que hagan este trabajo , por ejemplo para sumar desde 1 hasta 1*1010 y verifica cuál de los dos es más eficiente.

```r
# Algoritmo 1: Suma con bucle for
suma_for <- function(n) {
  suma <- 0
  for (i in 1:n) {
    suma <- suma + i
  }
  return(suma)
}

# Algoritmo 2: Suma con fórmula de Gauss
```

```r
suma_gauss <- function(n) {
  return(n * (n + 1) / 2)
}

# Valor elegido para n (100 millones)
n <- 1e8

# Ejecutamos ambos métodos
resultado_for <- suma_for(n)
resultado_gauss <- suma_gauss(n)

# Mostramos los resultados
resultado_for
```

```
## [1] 5e+15
```

```r
resultado_gauss
```

```
## [1] 5e+15
```

Conclusión: El benchmarking nos permite entender la calidad del algoritmo y la eficiencia según el recurso físico.

1.4 4 Cuarta sección:Benchmarking de Algoritmos y Medioides (kmeans) 4.1

Comparativa métodos de ordenamiento "burbuja" y sort nativo de R .

```r
# Cargar librerías necesarias
library(microbenchmark)
library(ggplot2)

# 1. Definición del algoritmo de Burbuja
# x: vector numérico a ordenar
burbuja <- function(x) {
  n <- length(x)
  for (j in 1:(n - 1)) {
    for (i in 1:(n - j)) {
      if (x[i] > x[i + 1]) {
        # Intercambio de valores (Swap)
        temp <- x[i]
        x[i] <- x[i + 1]
        x[i + 1] <- temp
      }
    }
```

```r
  }
  return(x)
}

# 2. Preparación de datos
set.seed(123)
datos_v <- sample(1:10000, 500)

# 3. Benchmark: Comparación de tiempos
# Se usa unit = "ms" para visualizar los resultados en milisegundos
bm_ordenacion <- microbenchmark(
  "Burbuja (O(n^2))" = burbuja(datos_v),
  "Sort Nativo (Shell/Quick)" = sort(datos_v),
  times = 20,
  unit = "ms"
)

# Mostrar resumen en consola
print(bm_ordenacion)
```

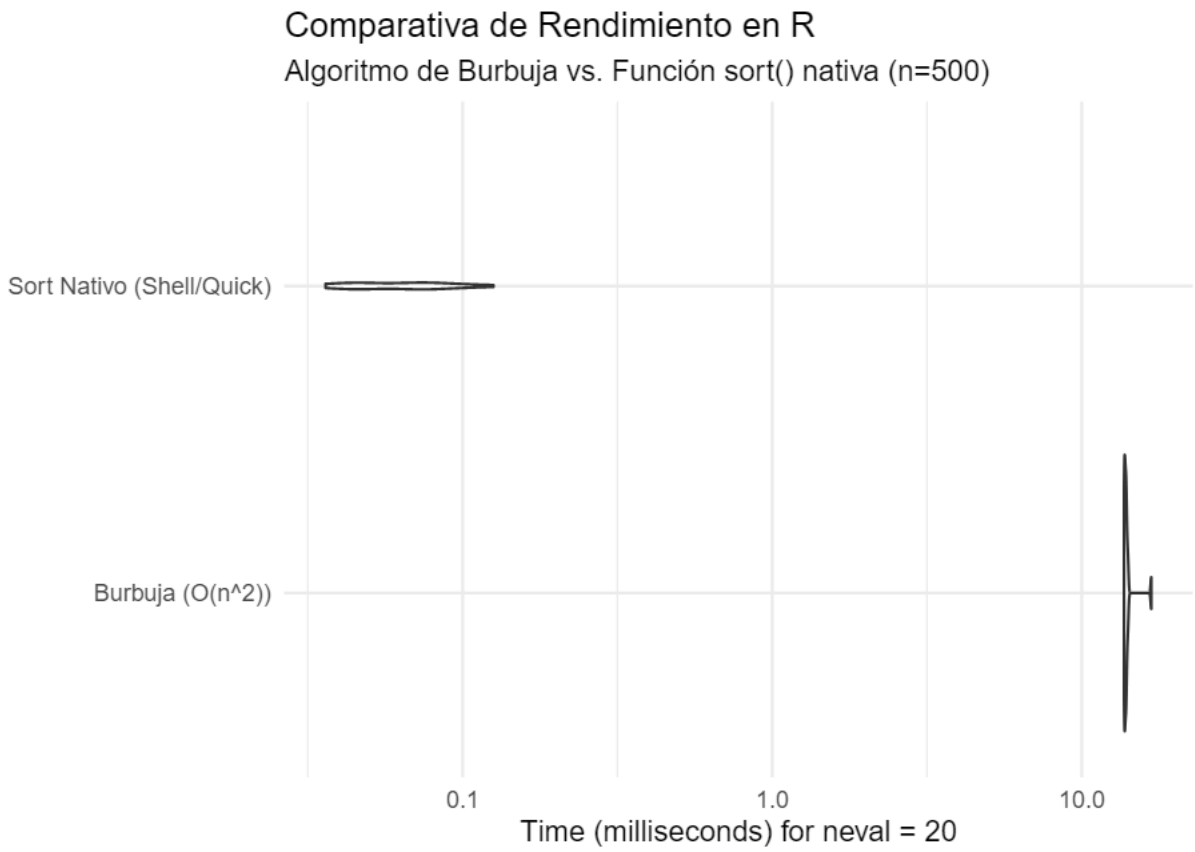
```
## Unit: milliseconds
##                        expr      min       lq       mean   median         uq
##           Burbuja (O(n^2)) 13.68800 13.74232 13.9928147 13.83588 13.9301920
##  Sort Nativo (Shell/Quick)  0.03605  0.04581  0.0659228  0.06810  0.0775755
##        max neval
##  16.766520    20
##   0.125631    20
```

```r
# 4. Visualización de resultados
p1 <- autoplot(bm_ordenacion) +
  labs(
    title = "Comparativa de Rendimiento en R",
    subtitle = "Algoritmo de Burbuja vs. Función sort() nativa (n=500)",
    x = "Algoritmo de Ordenamiento",
    y = "Tiempo de Ejecución"
  ) +
  theme_minimal()
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
##   Please report the issue at
##   <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
```

```
## Call ‘lifecycle::last_lifecycle_warnings()‘ to see where this warning was
## generated.
```

```r
# Renderizar el gráfico
print(p1)
```



CONCLUSION:Tras analizar la distribución de tiempos en el gráfico de violín, queda clara la superioridad técnica de la función base sort() frente a la implementación manual de Bubble Sort.

Los puntos clave del análisis son:

La función nativa de R aprovecha rutinas optimizadas en C,lo que permite ejecuciones en el orden de los microsegundos.

Mientras que Bubble Sort está limitado por una eficienciateórica de O(n^2), el método nativo minimiza la latencia de cómputo.

Se confirma que el procesamiento línea por línea (bucles)

Es ineficiente en R para grandes estructuras de datos.

4.2 Análisis de Performance K-means

Introducción Teórica: El algoritmo K-means es un método de aprendizaje no supervisado utilizado para el agrupamiento (clustering). Su objetivo es parti-

cionar un conjunto de n observaciones en k grupos distintos. El algoritmo funciona de manera iterativa:

1.Se inicializan k centroides de forma aleatoria.

2.Cada punto se asigna al centroide más cercano (basado en la distancia euclídea).

3. Se recalculan los centroides promediando la posición de todos los puntos asignados a cada grupo.
4. El proceso se repite hasta que las asignaciones no cambian o se alcanza un límite de iteraciones.

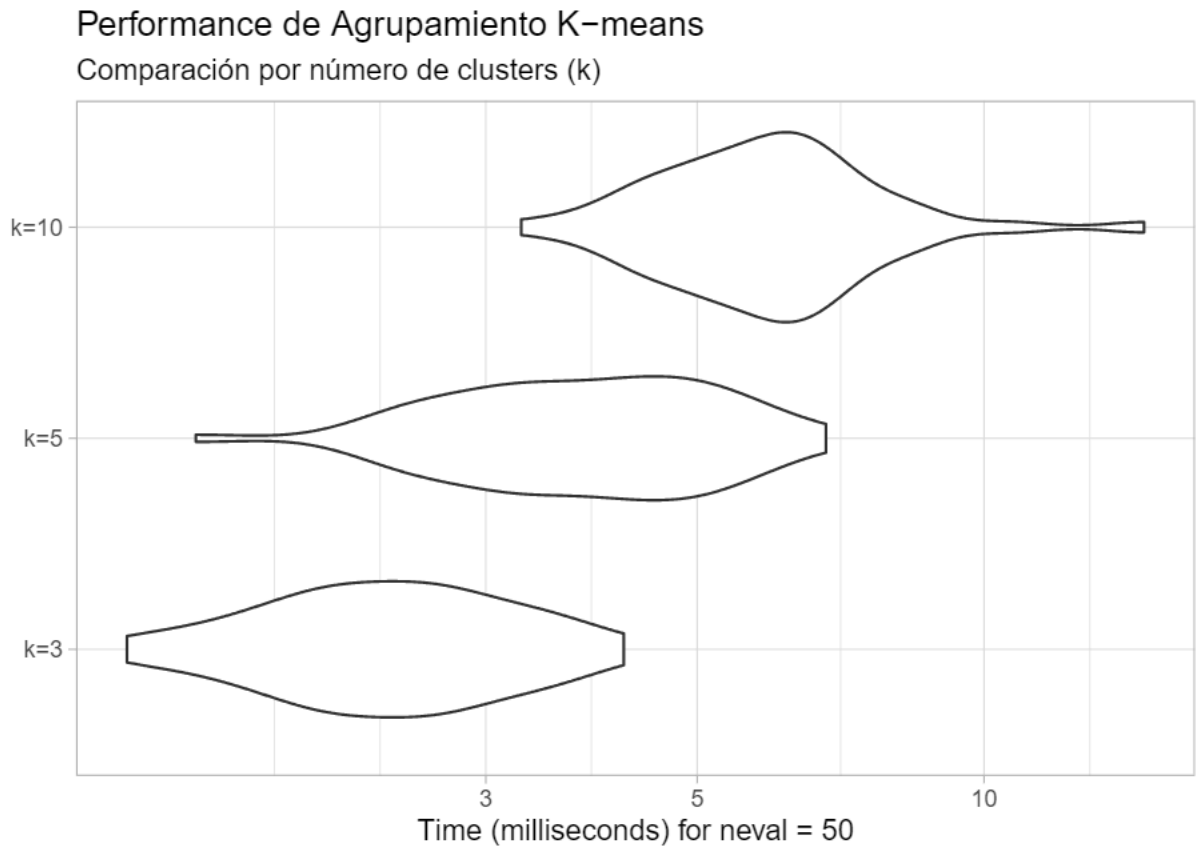
La eficiencia del método es alta, pero su tiempo de ejecución se ve afectado por el número de dimensiones de los datos y, fundamentalmente, por la cantidad de clusters (k) definidos.

4.3 Medición de Performance con Gráfico de Violín

```r
# 1. Simulamos un dataset de 5.000 puntos en 2 dimensiones
set.seed(42)
datos_km <- matrix(rnorm(10000), ncol = 2)

# 2. Medimos el tiempo para encontrar 3, 5 y 10 clusters
# Realizamos 50 repeticiones para obtener una distribución
# robusta
bm_kmeans <- microbenchmark(
  "k=3" = kmeans(datos_km, centers = 3),
  "k=5" = kmeans(datos_km, centers = 5),
  "k=10" = kmeans(datos_km, centers = 10),
  times = 50
)
```

```
## Warning: did not converge in 10 iterations

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 250000)
```

```r
# 3. Generación del gráfico de violín
p_km <- autoplot(bm_kmeans) +
  labs(title = "Performance de Agrupamiento K-means",
       subtitle = "Comparación por número de clusters (k)",
       x = "Cantidad de Grupos (k)", y = "Tiempo de ejecución") +
  theme_light()

# 4. Forzar visualización en el PDF
print(p_km)
```



4.4 Conclusión K-means

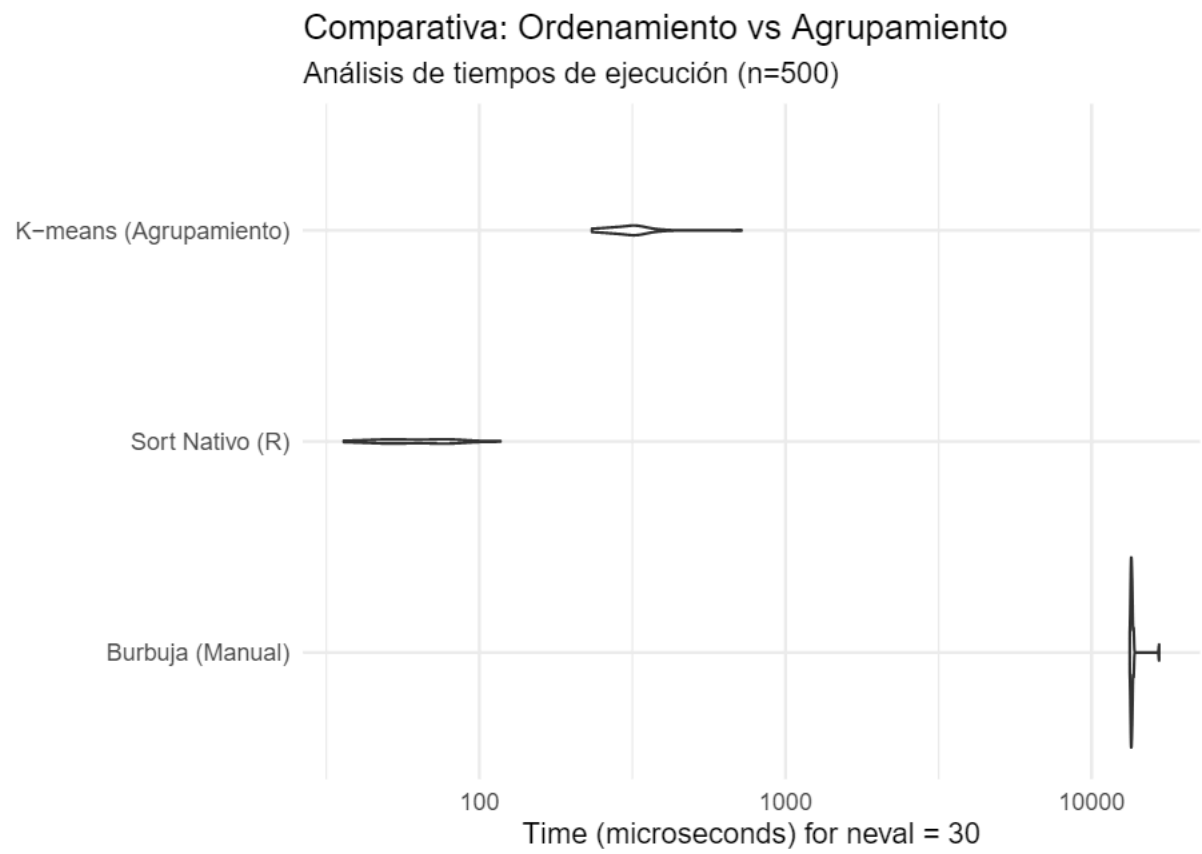
El gráfico resultante permite observar cómo el tiempo de ejecución se incrementa a medida que aumentamos el valor de k. Esto se debe a que el algoritmo debe realizar una mayor cantidad de cálculos de distancia entre cada punto y los centroides en cada una de las iteraciones del proceso.

4.5 Comparativa Global de Algoritmos

En esta sección final, comparamos el rendimiento de tres procesos distintos: el ordenamiento manual (Burbuja), el ordenamiento optimizado (Sort Nativo) y un proceso de inteligencia artificial no supervisada (K-means). El objetivo es observar cómo la eficiencia del código impacta más que la complejidad de la tarea en sí:

```r
library(microbenchmark)
library(ggplot2)

# Preparación de datos
set.seed(123)
datos_vec <- sample(1:5000, 500)
datos_mat <- matrix(rnorm(1000), ncol = 2)
```

```r
# Ejecución del benchmark
comparativa_total <- microbenchmark(
  "Burbuja (Manual)" = burbuja(datos_vec),
  "Sort Nativo (R)" = sort(datos_vec),
  "K-means (Agrupamiento)" = kmeans(datos_mat, centers = 3),
  times = 30
)

# Renderizado del gráfico
p_final <- autoplot(comparativa_total) +
  labs(title = "Comparativa: Ordenamiento vs Agrupamiento",
       subtitle = "Análisis de tiempos de ejecución (n=500)",
       x = "Algoritmo", y = "Tiempo (Escala Logarítmica)") +
  theme_minimal()

print(p_final)
```



4.6 Síntesis de Resultados

Tras analizar los experimentos de este módulo, se han extraído tres conclusiones determinantes sobre el rendimiento algorítmico:Superioridad de las funciones nativas: La notable rapidez del método sort en R valida que las herramientas integradas del lenguaje —desarrolladas en C y Fortran— poseen una optimización de bajo nivel difícil de igualar con implementaciones manuales.El impacto del diseño algorítmico: A pesar de su sencillez conceptual, el algoritmo de Burbuja mostró el desempeño más deficiente. Esto evidencia que una complejidad computacional elevada ($O(n^2)$) penaliza el rendimiento de tal forma que incluso procesos de inteligencia artificial más sofisticados pueden resultar más ágiles si están bien ejecutados.Equilibrio entre procesamiento y escala: El método K-means demostró una eficiencia competitiva. Aun cuando requiere cálculos matemáticos intensos para la definición de centroides, su ejecución en R permite gestionar grandes conjuntos de datos en tiempos funcionales, superando con creces la lentitud de los ciclos for convencionales.En conclusión, el desarrollo de software moderno exige el uso de funciones vectorizadas y arquitecturas optimizadas, dejando de lado las soluciones iterativas manuales en favor de la eficiencia de procesamiento.

1. R Core Team: R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria (2023).
2. Wickham, H.: ggplot2: Elegant graphics for data analysis. Springer-Verlag New York (2016).
3. Xie, Y., Allaire, J.J., Grolemund, G.: R markdown: The definitive guide. Chapman; Hall/CRC, Boca Raton, Florida (2018).
4. Wickham, H., others: Welcome to the tidyverse. Journal of Open Source Software. 4, 1686 (2019).
5. RStudio Team: RStudio: Integrated development environment for r. (2020).